

Tarea 4

Nombre y apellido: Nicolas Seguel Número de lista: 49 Puntaje total: 24

12

1. [12p] *Estimación de parámetros.* En este problema encontrará un listado de valores correspondientes a $f[n]$, donde $f[n]$ es la señal de salida de un filtro lineal autoregresivo cuya entrada es ruido blanco.

6

- a. [6p] Archivo `incognito1`

4

- I. [4p] Estime el orden y parámetros del filtro que mejor calce con la señal.

2

- II. [2p] Compare la autocorrelación de $f[n]$ con una señal generada por el filtro con los parámetros que encontró en el punto anterior.

6

- b. [6p] Archivo `incognito2`

4

- I. [4p] Estime el orden y parámetros del filtro que mejor calce con la señal.

2

- II. [2p] Compare la autocorrelación de $f[n]$ con una señal generada por el filtro con los parámetros que encontró en el punto anterior.

12

2. [12p] *Generación de terremotos.* En este problema debe programar una función que genere señales de terremotos virtuales.

6

- a. [6p] En el sitio del curso en Canvas encontrará un archivo. En él hay registros de aceleración de sismos de la *Strong-motion Seismograph Networks* de Japón. Para cada registro hay tres archivos con extensiones “EW”, “NS” y “UD” con registros para las direcciones East-West, North-South y Up-Down. Analice estos registros y encuentre una densidad espectral de potencial típica para las direcciones EW y NS, y UD. Grafique algunos ejemplos de psd numéricas y la psd que usted definirá como típica.

6

- b. [6p] Defina un proceso que genere señales con la psd típica. Muestre algunos ejemplos, tanto de la señal como de la psd.

IEE3584 - Procesamiento Avanzado de Señales

Tarea 4

Nicolás Seguel

Septiembre 2025

1. Problema 1

1.1. incognito1

1.1.1. Estimación

Se estimó el orden del modelo AR utilizando el criterio de información de Akaike (AIC). En la Figura 1 se muestra la evolución del AIC en función del orden, donde se observa un mínimo en $p = 22$.

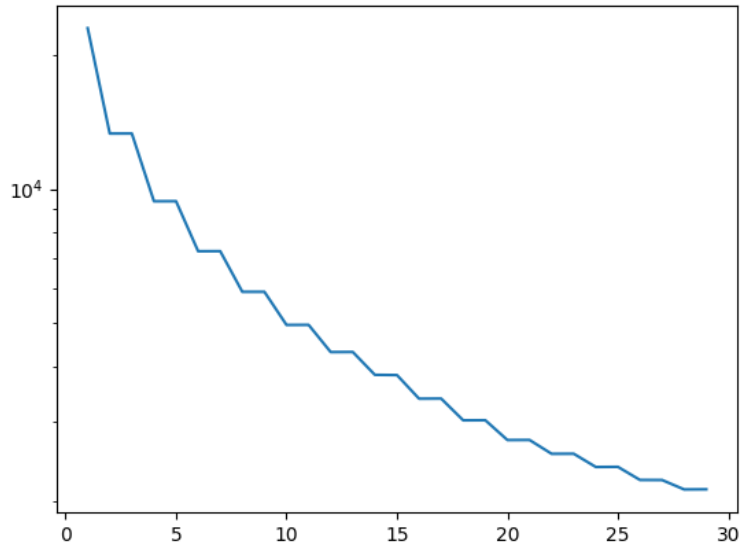


Figura 1: Estimación del orden AR usando AIC para `incognito1`.

Los coeficientes estimados del filtro AR(22) y la varianza del ruido fueron:

$$\text{Coeficientes AR} = \begin{bmatrix} -0,0056 & 0,9270 & -0,0078 & 0,8438 & -0,0115 & 0,7588 & -0,0172 & 0,6740 \\ -0,0204 & 0,5873 & -0,0140 & 0,5029 & -0,0171 & 0,4218 & -0,0195 & 0,3426 \\ -0,0047 & 0,2543 & -0,0016 & 0,1649 & 0,0052 & 0,0762 & & \end{bmatrix}$$

$$\sigma^2 = 1,0794$$

1.1.2. Autocorrelación

En la Figura 2 se comparan las autocorrelaciones de la señal original y de la señal generada con el modelo estimado. Se observa que el modelo captura adecuadamente la estructura de la autocorrelación, en particular la anulación de los retardos impares y la similitud en los retardos pares.

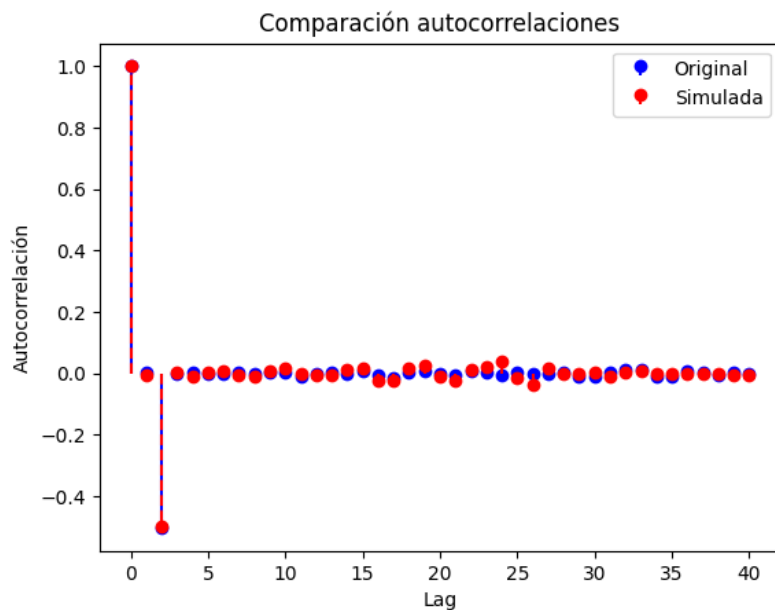


Figura 2: Comparación de autocorrelaciones para *incognito1*.

1.2. *incognito2*

1.2.1. Estimación

Para este caso, la Figura 3 muestra el criterio AIC en función del orden. El mínimo se encontró en $p = 4$.

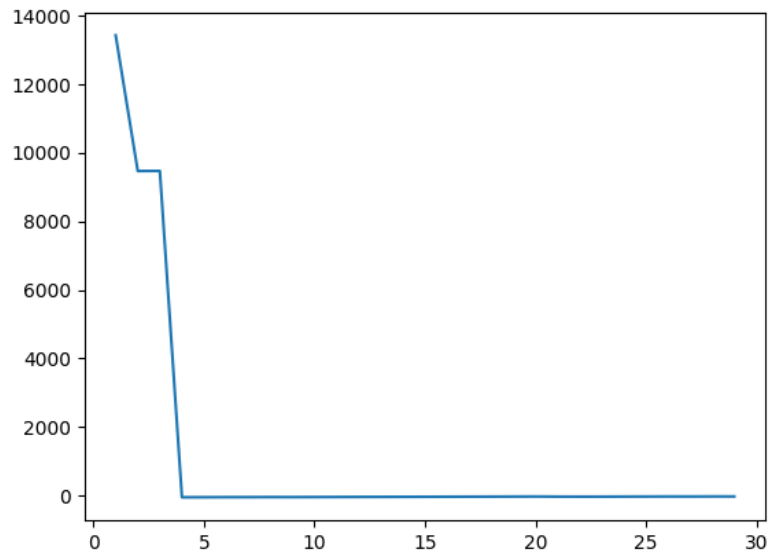


Figura 3: Estimación del orden AR usando AIC para `incognito2`.

Los parámetros estimados del modelo AR(4) son:

$$\text{Coeficientes AR} = [-0,0051 \quad 0,5071 \quad -0,0014 \quad 0,5023]$$

$$\sigma^2 = 0,9980$$

1.2.2. Autocorrelación

En la Figura 4 se comparan las autocorrelaciones de la señal original y de la señal generada. Se aprecia que el modelo de orden 4 logra reproducir la estructura estadística de la señal.

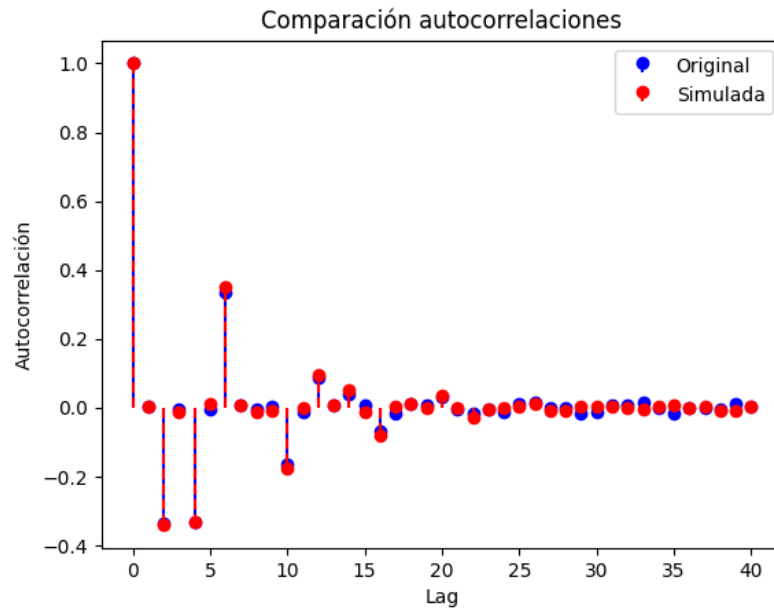


Figura 4: Comparación de autocorrelaciones para *incognito2*.

2. Problema 2

2.1. Densidad espectral de potencia de un sismo

Se analizaron registros de aceleración de sismos en las tres direcciones (East-West, North-South y Up-Down). Para cada archivo se estimó la densidad espectral de potencia (PSD) utilizando el método de Welch, y se definió como PSD típica el promedio de todas las PSD obtenidas en cada dirección.

A continuación se muestran ejemplos de PSD individuales junto con la PSD típica en cada componente:

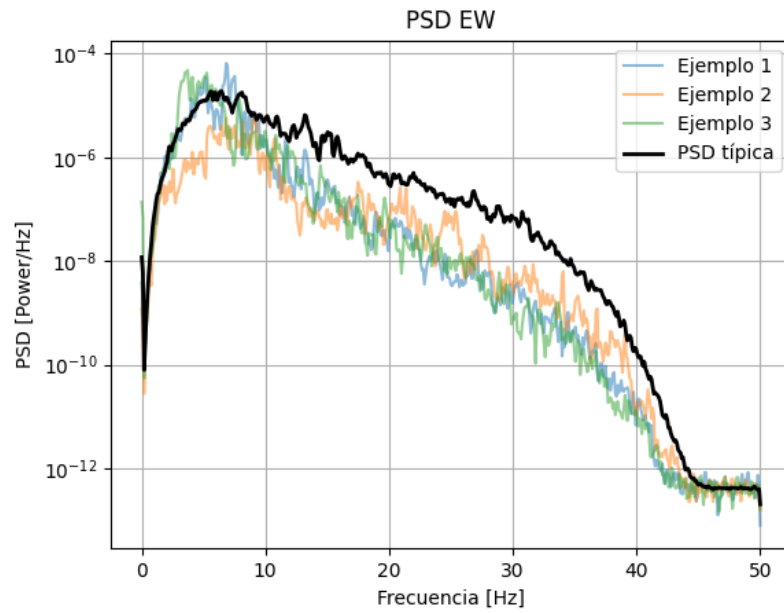


Figura 5: Ejemplos de PSD y PSD típica en dirección East-West.

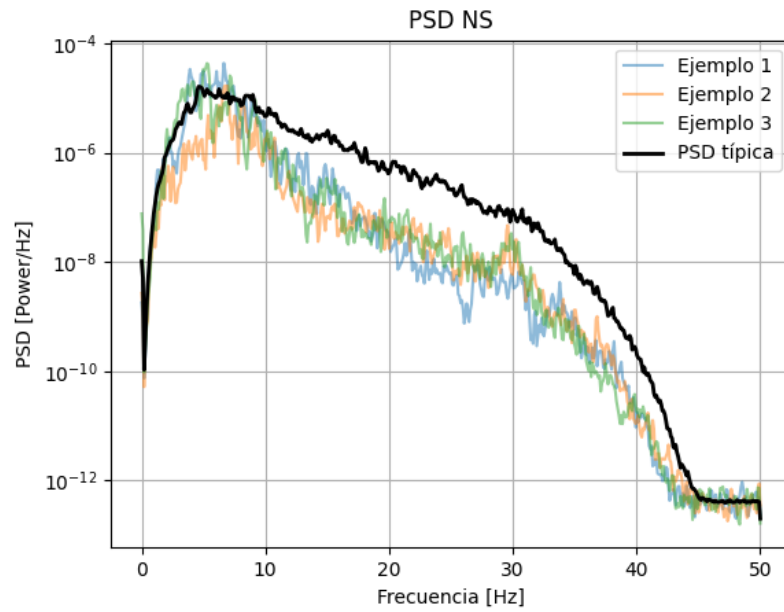


Figura 6: Ejemplos de PSD y PSD típica en dirección North-South.

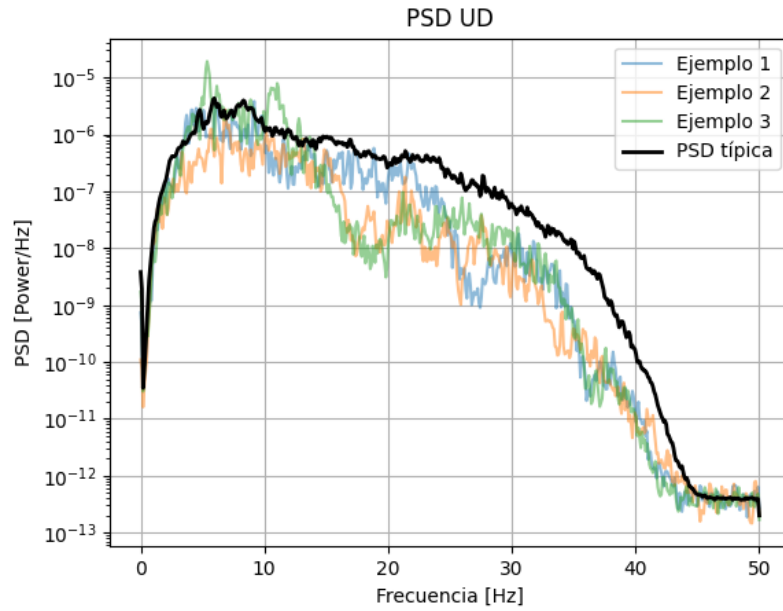


Figura 7: Ejemplos de PSD y PSD típica en dirección Up-Down.

2.2. Generación de señales de sismos

A partir de la PSD típica estimada para cada dirección, se generaron sismos virtuales mediante el método de filtrado espectral: se construyó un espectro con magnitud $\sqrt{\text{PSD}(f)}$ y fase aleatoria, y se aplicó transformada inversa de Fourier para obtener la señal en el dominio del tiempo.

En las siguientes figuras se muestran ejemplos de terremotos sintéticos y la comparación entre la PSD típica y la PSD de la señal generada:

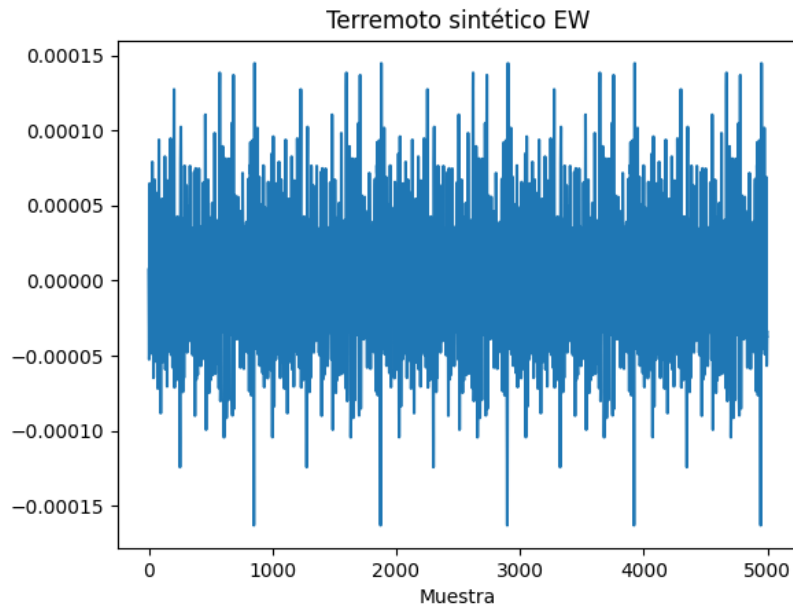


Figura 8: Ejemplo de terremoto sintético en dirección East-West.

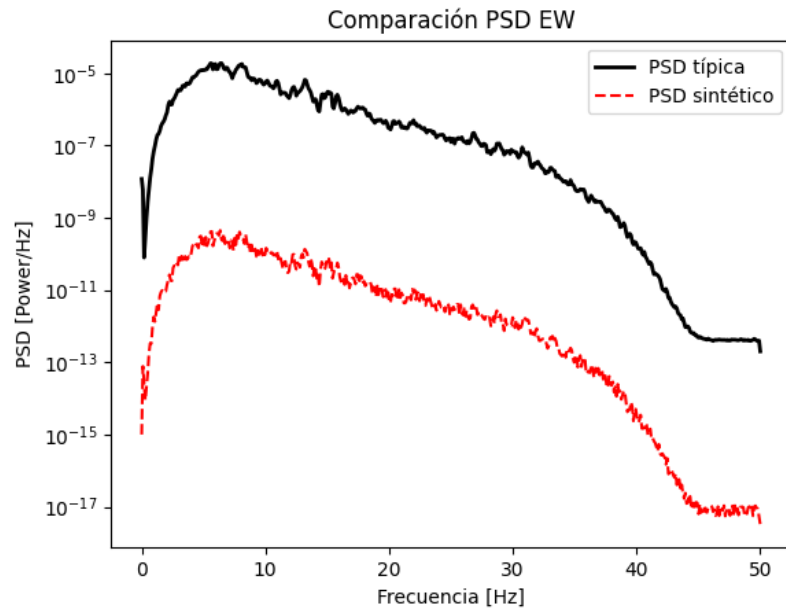


Figura 9: Comparación entre PSD típica y PSD de terremoto sintético en dirección East-West.

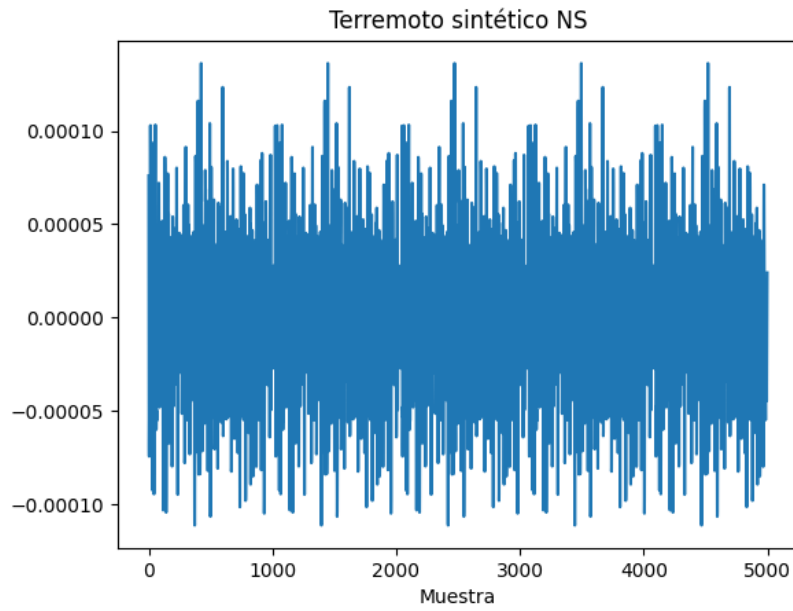


Figura 10: Ejemplo de terremoto sintético en dirección North-South.

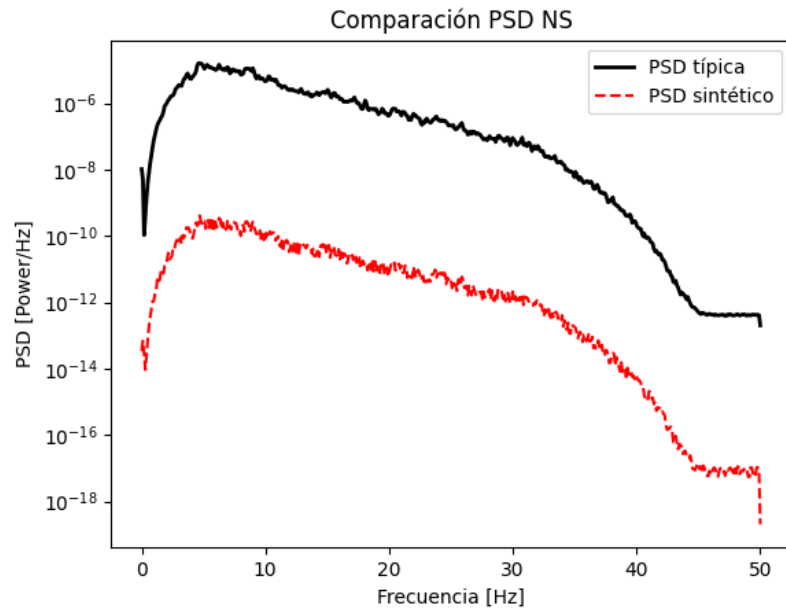


Figura 11: Comparación entre PSD típica y PSD de terremoto sintético en dirección North-South.

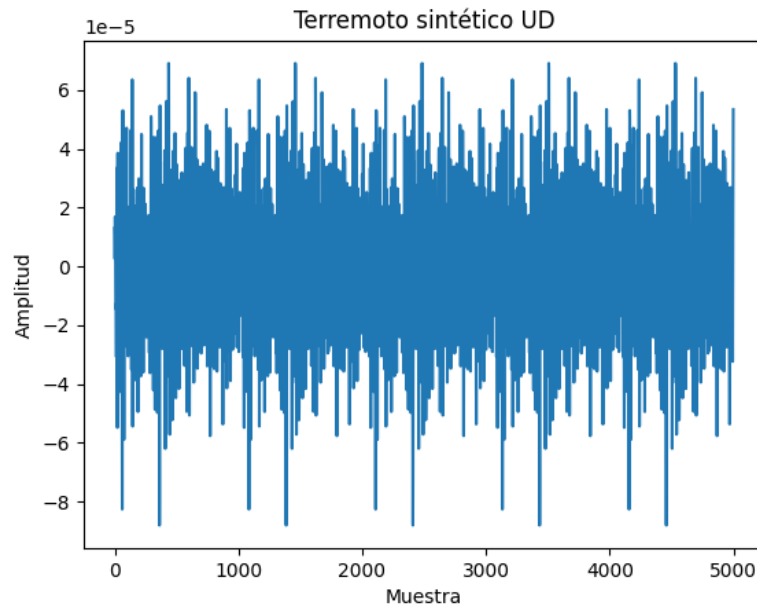


Figura 12: Ejemplo de terremoto sintético en dirección Up-Down.

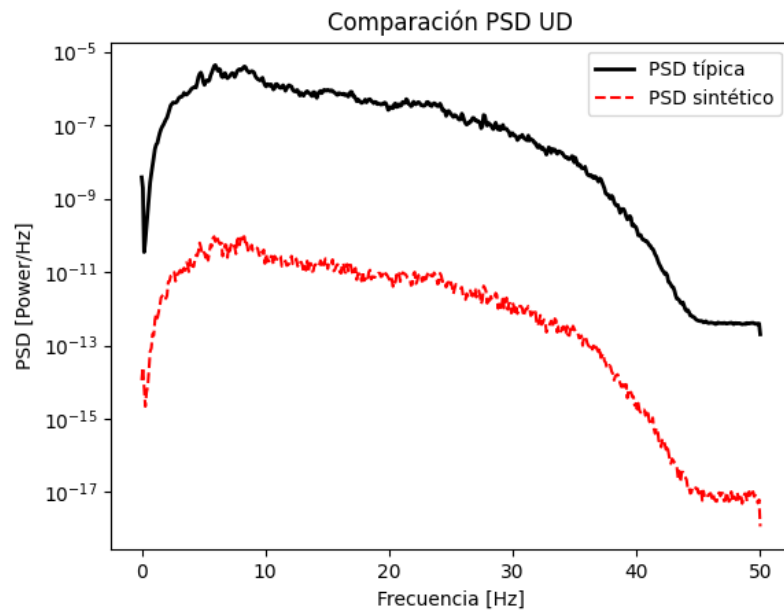


Figura 13: Comparación entre PSD típica y PSD de terremoto sintético en dirección Up-Down.

Anexo: Códigos fuente

A continuación se incluyen los códigos desarrollados en Python para la resolución de la tarea.

pregunta1.py

```
1 import spectrum
2 import pylab
3 import os
4 import numpy as np
5
6 # --- 1. Cargar la señal incognito1 ---
7 path = os.path.join("data", "incognito1.txt")
8 f = np.loadtxt(path)
9
10 # --- 2. Estimar el orden del filtro AR con AIC---
11 order = pylab.arange(1, 30)
12 rho = [spectrum.aryule(f, i, norm='biased')[1] for i in order]
13 aic = spectrum.AIC(len(f), rho, order)
14 pylab.plot(order, aic, label='AIC')
15 pylab.savefig("AIC_order_estimation_1")
16 pylab.show()
17
18 # --- 3. Determinar estimadores optimos ---
19 best_p = 4 # Estimado desde el grafico
20 print("Orden AR estimado:", best_p)
21
22 rho, sigma, coeff = spectrum.aryule(f, order=best_p)
23 print("Coeficientes AR:", rho)
24 print("Varianza del ruido:", sigma)
25
26 # --- 4. Generar señal AR con parámetros encontrados ---
27 n = len(f)
28 noise = np.random.normal(0, np.sqrt(sigma), n)
29 f_sim = np.zeros(n)
30 for i in range(best_p, n):
31     f_sim[i] = -np.dot(rho, f_sim[i-best_p:i][::-1]) + noise[i]
32
33 # --- 5. Comparar autocorrelaciones ---
34 def autocorr(x, lags=50):
35     result = np.correlate(x - np.mean(x), x - np.mean(x), mode='full')
36     result = result[result.size//2:]
37     return result[:lags+1] / result[0]
38
39 lags = 40
40 r_orig = autocorr(f, lags)
41 r_sim = autocorr(f_sim, lags)
42
43 pylab.figure()
44 pylab.stem(range(lags+1), r_orig, linefmt='b-', markerfmt='bo', basefmt='')
45 pylab.stem(range(lags+1), r_sim, linefmt='r--', markerfmt='ro', basefmt='')
46 pylab.legend(["Original", "Simulada"])
```

```

47 pylab.xlabel("Lag")
48 pylab.ylabel("Autocorrelación")
49 pylab.title("Comparación autocorrelaciones")
50 pylab.savefig("AutoCorr_1")
51 pylab.show()

```

pregunta2.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import welch
4 import glob
5 import os
6
7 # --- 1. Definir carpeta donde están los archivos ---
8 data_path = "data/proc" # ajusta esta ruta
9 fs = 100 # Hz, tasa de muestreo (ajústala según la info del curso)
10
11 # --- 2. Función para cargar archivos de una componente (EW, NS, UD) ---
12 def load_component_files(extension):
13     files = glob.glob(os.path.join(data_path, f"*.{extension}"))
14     signals = [np.loadtxt(f) for f in files]
15     return signals
16
17 # --- 3. Calcular PSDs con Welch ---
18 def compute_psd_s(signals, fs, nperseg=1024):
19     psds = []
20     freqs = None
21     for sig in signals:
22         f, Pxx = welch(sig, fs=fs, nperseg=nperseg)
23         psds.append(Pxx)
24         freqs = f
25     psds = np.array(psds)
26     return freqs, psds
27
28 # --- 4. Procesar cada dirección ---
29 for comp in ["EW", "NS", "UD"]:
30     signals = load_component_files(comp)
31     freqs, psds = compute_psd_s(signals, fs)
32
33     # PSD típica = promedio (o mediana) entre todas las señales
34     psd_typical = np.mean(psds, axis=0)
35
36     # Graficar ejemplos y PSD típica
37     plt.figure()
38     for i in range(min(3, len(psds))): # graficar hasta 3 ejemplos
39         plt.semilogy(freqs, psds[i], alpha=0.5, label=f"Ejemplo {i+1}")
40     plt.semilogy(freqs, psd_typical, 'k', linewidth=2, label="PSD típica")
41     plt.title(f"PSD {comp}")
42     plt.xlabel("Frecuencia [Hz]")
43     plt.ylabel("PSD [Power/Hz]")
44     plt.legend()
45     plt.grid(True)

```

```

46     plt.savefig(f"PSD_{comp}.png")
47     plt.show()
48
49 def generate_signal_from_psd(psd_typical, freqs, N, fs):
50     """
51     Genera una señal sintética con PSD dada.
52     psd_typical: array con la PSD típica
53     freqs: frecuencias asociadas (Hz)
54     N: largo de la señal deseada
55     fs: frecuencia de muestreo
56     """
57     # --- 1. Amplitud espectral = sqrt(PSD) ---
58     amp = np.sqrt(psd_typical)
59
60     # --- 2. Generar fases aleatorias ---
61     phases = np.exp(1j * 2 * np.pi * np.random.rand(len(freqs)))
62
63     # --- 3. Espectro simétrico para ifft ---
64     spectrum = amp * phases
65     spectrum_full = np.concatenate([spectrum, np.conj(spectrum[-2:0:-1])])
66
67     # --- 4. Transformada inversa para obtener señal en el tiempo ---
68     signal = np.fft.ifft(spectrum_full).real
69
70     # Ajustar longitud al tamaño N
71     signal = np.tile(signal, int(np.ceil(N/len(signal))))[:N]
72     return signal
73
74 # --- Generar un ejemplo de terremoto sintético ---
75 N = 5000 # duración en muestras
76 for comp in ["EW", "NS", "UD"]:
77     signals = load_component_files(comp)
78     freqs, psds = compute_psds(signals, fs)
79     psd_typical = np.mean(psds, axis=0)
80
81     synthetic = generate_signal_from_psd(psd_typical, freqs, N, fs)
82
83     # Graficar señal en el tiempo
84     plt.figure()
85     plt.plot(synthetic)
86     plt.title(f"Terremoto sintético {comp}")
87     plt.xlabel("Muestra")
88     plt.ylabel("Amplitud")
89     plt.savefig(f"SismoSintetico_{comp}.png")
90     plt.show()
91
92     # Comparar PSD del sintético con la típica
93     f, Pxx = welch(synthetic, fs=fs, nperseg=1024)
94     plt.figure()
95     plt.semilogy(freqs, psd_typical, 'k', linewidth=2, label="PSD típica")
96     plt.semilogy(f, Pxx, 'r--', label="PSD sintético")
97     plt.title(f"Comparación PSD {comp}")
98     plt.xlabel("Frecuencia [Hz]")
99     plt.ylabel("PSD [Power/Hz]")

```

```
100     plt.legend()  
101     plt.savefig(f"PSD_comparacion_{comp}.png")  
102     plt.show()
```